

ASSIGNMENT

11. QUESTION

A government database stores the following identification numbers:

A102, A25, A7, B100, B12, A120, B3, A45

a) Implement a BST to organise the identification numbers and display its inorder traversal. Analyse the resulting tree structure.

b) Compare the performance of BST Search and Linear Search for selected

identification numbers. Record the number of comparisons.

c) Analyse how varying key lengths and insertion order affect the BST height and search performance. Compare the observed performance with the theoretical best-case and worst-case complexities.

Suggest a suitable approach for maintaining efficient searches as the database grows.

PROGRAM

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 struct Node {
6     char key[20];
7     struct Node *left;
8     struct Node *right;
9 };
10
11 struct Node* createNode(char key[]) {
12     struct Node *newNode;
13
14     newNode = (struct Node*)malloc(sizeof(struct Node));
15
16     strcpy(newNode->key, key);
17     newNode->left = NULL;
18     newNode->right = NULL;
19
20     return newNode;
21 }
22
23 struct Node* insert(struct Node *root, char key[]) {
24
25     if (root == NULL)
26         return createNode(key);
```

```
27
28     if (strcmp(key, root->key) < 0)
29         root->left = insert(root->left, key);
30
31     else if (strcmp(key, root->key) > 0)
32         root->right = insert(root->right, key);
33
34     return root;
35 }
36
37 void inorder(struct Node *root) {
38
39     if (root != NULL) {
40         inorder(root->left);
41         printf("%s ", root->key);
42         inorder(root->right);
43     }
44 }
45
46 int bstSearch(struct Node *root, char key[]) {
47
48     int comparisons = 0;
49
50     while (root != NULL) {
51
52         comparisons++;
```

```

53
54     int result = strcmp(key, root->key);
55
56     if (result == 0)
57         return comparisons;
58
59     else if (result < 0)
60         root = root->left;
61
62     else
63         root = root->right;
64 }
65
66 return comparisons;
67 }
68
69 int linearSearch(char data[][20], int n, char key[]) {
70
71     int comparisons = 0;
72
73     for (int i = 0; i < n; i++) {
74
75         comparisons++;
76
77         if (strcmp(data[i], key) == 0)

```

```

78             return comparisons;
79     }
80
81     return comparisons;
82 }
83
84 int main() {
85
86     char data[][20] = {
87         "A102", "A25", "A7", "B100",
88         "B12", "A120", "B3", "A45"
89     };
90
91     int n = 8;
92
93     struct Node *root = NULL;
94
95     for (int i = 0; i < n; i++) {
96         root = insert(root, data[i]);
97     }
98
99     printf("Inorder Traversal:\n");
100    inorder(root);
101
102    char key1[] = "A120";
103    char key2[] = "A7";

```

```

104     char key3[] = "B3";
105
106     printf("\n\nSearch Key: %s\n", key1);
107     printf("BST Search Comparisons: %d\n",
108           bstSearch(root, key1));
109     printf("Linear Search Comparisons: %d\n",
110           linearSearch(data, n, key1));
111
112     printf("\n\nSearch Key: %s\n", key2);
113     printf("BST Search Comparisons: %d\n",
114           bstSearch(root, key2));
115     printf("Linear Search Comparisons: %d\n",
116           linearSearch(data, n, key2));
117
118     printf("\n\nSearch Key: %s\n", key3);
119     printf("BST Search Comparisons: %d\n",
120           bstSearch(root, key3));
121     printf("Linear Search Comparisons: %d\n",
122           linearSearch(data, n, key3));
123
124     return 0;
125 }

```

OUT PUT

```

Inorder Traversal:
A102 A120 A25 A45 A7 B100 B12 B3

Search Key: A120
BST Search Comparisons: 3
Linear Search Comparisons: 6

Search Key: A7
BST Search Comparisons: 3
Linear Search Comparisons: 3

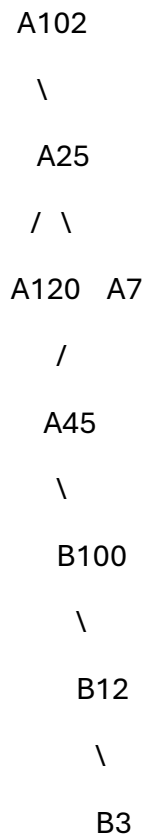
Search Key: B3
BST Search Comparisons: 6
Linear Search Comparisons: 7

```

Input Data

A102, A25, A7, B100, B12, A120, B3, A45

BST Representation and Inorder Traversal



Inorder Traversal:

A102 → A120 → A25 → A45 → A7 → B100 → B12 → B3

Comparison of BST Search and Linear Search

Search Key	BST Search Comparisons	Linear Search Comparisons
------------	------------------------	---------------------------

A120	3	6
------	---	---

A7	3	3
----	---	---

B3	6	7
----	---	---

Observation:

BST Search requires fewer comparisons for A120 and B3 compared with Linear Search for the given input. For A7, both methods require 3 comparisons.

Complexity Analysis

BST Search

- **Best-case time complexity:** $O(1)$
- **Average-case time complexity:** $O(\log n)$
- **Worst-case time complexity:** $O(n)$
- **Space complexity:** $O(n)$

Linear Search

- **Best-case time complexity:** $O(1)$
- **Average-case time complexity:** $O(n)$
- **Worst-case time complexity:** $O(n)$
- **Space complexity:** $O(1)$

Effect of Key Length and Insertion Order

The length of the identification keys affects the cost of comparing two keys because the keys are compared character by character. The insertion order affects the height of the BST. A balanced BST has a smaller height and provides faster searches, while an unbalanced or skewed BST can increase the search time to $O(n)$.

For the given data, the observed BST search comparisons depend on the position of each identification number in the tree. The theoretical best case is $O(1)$, while the worst case is $O(n)$.

Suitable Approach

For a growing database, a **self-balancing BST such as an AVL Tree or Red-Black Tree** can be used to maintain efficient search performance by keeping the tree height approximately balanced.

Conclusion

The given identification numbers were successfully organised using a Binary Search Tree. The inorder traversal produced the identification numbers in lexicographic order.

The comparison between BST Search and Linear Search shows that the number of comparisons depends on the position of the identification number and the structure of the BST. The insertion order affects the height of the tree and therefore affects search performance.

A balanced BST provides efficient searching, while an unbalanced BST can have $O(n)$ worst-case search time. For a growing database, a self-balancing BST such as an AVL Tree or Red-Black Tree is a suitable approach for maintaining efficient searches